湖南科技大学计算机科学与工程学院

智能终端系统及应用开发

课程设计报告

基于 Android 的组件化短视频应用设计与实现

专业班级：2024 级物联网工程三班

姓　　名：李思民

学　　号：2405040303

指导教师：黄晶

设计时间：2026.10.5—2026.10.26

实验地点：逸夫楼 433

目录

一　实验题目

二　实验目的

三　总体设计

3.1 需求分析与功能边界

3.2 技术方案与模块划分

3.3 数据存储与业务流程

四　详细设计

4.1 主界面导航与组件路由

4.2 网络请求与首页分页

4.3 视频播放与生命周期管理

4.4 登录认证与互动功能

4.5 广场发现与相机功能

4.6 本地浏览记录

五　实验结果与分析

5.1 功能核对与测试设计

5.2 界面截图与运行结果记录

5.3 问题分析与改进方向

六　小结与心得体会

一　实验题目

基于 Android 的组件化短视频应用设计与实现。项目工程名称为 shortVideo，围绕视频内容浏览与播放，组织首页、广场、发现和个人中心四个主要入口，形成“浏览内容—进入播放—登录互动—收藏与历史回看”的使用流程。

客户端采用 Java 与 Kotlin 混合开发，使用 MVVM、DataBinding 和组件路由组织页面，通过 Retrofit 与 OkHttp 对接既有远程接口，并利用 Room 保存本地浏览记录。设计重点为 Android 客户端的功能组织、数据交互和媒体资源管理，服务端接口由现有服务提供。

二　实验目的

掌握 Android 多页面应用的组织方法，将 Activity、Fragment、ViewPager2 和 RecyclerView 综合应用于内容浏览场景，理解页面切换、列表复用和生命周期之间的关系。

理解 MVVM 中界面、状态与数据访问的职责，使用 LiveData 与 DataBinding 传递数据，减少界面代码与网络逻辑的直接耦合。

掌握组件化工程的基本结构，利用 ARouter 完成跨模块页面跳转和参数传递，认识公共基础模块在复用和统一交互中的作用。

掌握视频播放的基础流程，使用 Media3 ExoPlayer 加载媒体地址，并根据页面生命周期暂停与释放资源，避免后台播放及资源泄漏。

掌握远程数据与本地持久化的协同方式，理解分页、登录令牌、浏览记录、异步数据库操作及错误状态处理。

形成以证据验证功能的习惯，对正常流程、异常输入、断网及登录状态变化分别设计测试，分析当前实现的不足。

三　总体设计

3.1 需求分析与功能边界

应用面向通过移动终端浏览短视频的用户。游客主要浏览推荐、日报、分类和视频详情；登录后可使用点赞、收藏、评论和个人资料管理。首页采用纵向视频卡片列表，点击后进入独立播放页，并非全屏上下滑动切换的视频流。微信登录、微信分享及视频拍摄发布链路不列入已完成功能，相机模块主要服

务于头像拍照。

3.2 技术方案与模块划分

工程以 app 为集成入口，业务模块依赖公共基础能力，通过路由访问其他业务页面。根工程 isModule=false 时，各业务模块以库的形式参与集成；切换组件模式时，公共构建脚本调整插件与清单文件，以支持模块调试。

层次或模块	主要职责
app	应用启动、底部四入口、Fragment 切换
library_base	页面基类、路由常量、用户管理、Room、工具类
network 与 video_data	Retrofit 请求封装、统一响应和业务数据模型
feature_home	推荐与日报容器，复用视频列表
feature_mediaplay	播放、互动、搜索、分类列表、收藏、浏览记录
feature_piazza 与 feature_find	广场图文、图片浏览、分类、主题与话题入口
feature_user 与 feature_camera	验证码登录、资料设置、头像拍照

工程编译配置为 compileSdk 35、targetSdk 35、minSdk 24，Java 编译目标为 11。核心依赖包括 Retrofit 2.11.0、OkHttp 4.12.0、Glide 4.16.0、Room 2.6.1、ARouter 1.5.2 和 Media3 1.0.0。版本描述以工程配置为准，用于说明本项目采用的开发环境。

【插图位置】插入分层架构图：顶部为四个主入口，中部为各 feature 模块，下部为 library_base、network、video_data；外侧标出远程接口与本地 Room。箭头说明路由、请求和存储关系。

图 3-1　系统总体架构

3.3 数据存储与业务流程

远程接口负责视频内容、用户资料和互动数据。用户管理器使用 EncryptedSharedPreferences 保存令牌和必要资料；浏览记录使用 Room 单独保存。视频媒体仍通过播放地址在线加载，浏览记录只保存索引与展示字段，不等于离线视频缓存。

主要数据流为：页面发起操作，ViewModel 组织状态，Model 请求 API，再将结果转换为 LiveData 更新界面。登录后广播登录事件，相关页面重新获取资料或互动状态，使个人中心与播放页保持一致。

四　详细设计

4.1 主界面导航与组件路由

MainActivity 将首页、广场、发现和我的入口映射到路由路径。首次切换时创建并添加对应 Fragment，后续以 show 和 hide 切换。该方式能够保留列表位置与页面数据，适合四个入口间频繁切换的场景；页面保留也意味着需要注意内存占用以及隐藏页面的后台工作。

路由常量集中定义在 ArouterPath 中，视频列表通过视频编号跳转播放页，目标页面通过参数注入获得编号，再请求完整详情。调用方不必直接引用目标 Activity 的具体类型，从而降低模块之间的耦合。

```
ARouter.getInstance()
    .build(ArouterPath.Video.ACTIVITY_MEDIA_PLAY)
    .withInt(ArouterPath.Video.KEY_VIDEO_ID, videoId)
    .navigation();
```

以上为路由调用方式的简化示意。工程公共基类承担 ViewModel 初始化、DataBinding 绑定和参数注入，并统一观察加载、提示及结束页面等状态，使业务页面集中处理自身逻辑。

4.2 网络请求与首页分页

网络层采用 Retrofit 定义接口、OkHttp 执行请求、Gson 解析 JSON。Java 回调封装与 Kotlin 协程封装目前并存，播放页采用 Kotlin 链路。统一响应包含 code、msg、time 和 data，列表数据进一步包含 count 与 list。各接口需要根据实际业务码判断成功，不能仅凭 HTTP 请求完成就视为业务成功。

首页由 ViewPager2 承载推荐和日报两页，列表复用 VideoListFragment。首次加载与下拉刷新从第一页请求，上拉加载请求后续页；刷新替换旧数据，加载更多追加数据。当前列表按每页十条组织，不足一页时关闭继续加载。异常处理需保留已有列表，并结束刷新动画，避免用户误以为请求仍在进行。

【插图位置】插入流程图：首次加载或刷新→重置页码→请求接口→判断

成功、空数据或失败→替换或追加列表→结束加载；另标出加载更多失败后的页码回退检查。

图 4-1　列表请求与界面更新流程

4.3 视频播放与生命周期管理

实际路由指向 MediaPlayActivityKotlinVersion。播放页接收视频编号后，由 ViewModel 请求详情，Activity 观察视频数据并将 video_file 交给播放器，同时刷新评论。简介与评论由 ViewPager2 组织，使媒体区域和信息区域在同一页面中协同展示。

MediaPlayerManager 对 ExoPlayer 进行单例封装，使用应用级 Context 创建实例，并绑定到 PlayerView。切换媒体时设置 MediaItem，完成准备后播放。页面销毁时停止播放、清空媒体并释放播放器，同时清除单例引用。

```
public void play(String url) {

    if (mPlayer != null) {

        MediaItem mediaItem = MediaItem.fromUri(url);

        mPlayer.setMediaItem(mediaItem);

        mPlayer.prepare();

        mPlayer.play();

    }

}
```

上述代码摘自 MediaPlayerManager.java。播放器调用简单，但稳定性取决于媒体地址、网络状态以及生命周期配合。当前 Activity 在 onStart 中允许播放，在 onPause 和 onStop 中暂停，在 onDestroy 中释放。对于短暂遮挡后只回到 onResume 的情况，仍需验证恢复策略，避免出现页面可见而视频未恢复的问题。

播放页采用 singleTask。再次点击视频卡片时，通过 onNewIntent 接收新编号并重新请求详情，复用现有页面。切换视频时不仅要更新媒体，还应核对评论、点赞数、收藏状态和推荐内容是否全部对应新视频，防止旧状态残留。

【插图位置】插入时序图：视频卡片→路由→播放页→ViewModel→接口→视频详情→播放器；在旁边标注 onPause 暂停、onDestroy 释放，以及 Room

写入浏览记录。

图 4-2　播放页的数据与资源流程

4.4 登录认证与互动功能

用户模块提供手机号验证码登录入口。登录成功后保存令牌与资料，后续受保护接口通过 token 请求头携带身份信息。退出登录清理本地会话，并通过 EventBus 通知相关页面刷新。令牌非空只表示客户端存在会话，是否有效仍需由服务端校验。

4.4 登录认证与互动功能续

点赞、收藏和评论操作由 ViewModel 判断登录状态。未登录时提示用户登录；登录后调用相应接口。点赞与取消点赞、收藏与取消收藏分别处理，返回结果驱动数量和状态更新。点赞接口与一般接口存在不同成功码，需要在业务层明确区分。

互动界面存在提前修改本地状态的处理方式。若网络失败，界面可能与服务端不一致，因此应补充失败回滚或在请求完成后重新拉取详情。评论输入弹窗与 Kotlin 播放链路仍有新旧 ViewModel 并存问题，后续应统一到页面共享的状态源，防止同一视频出现多份互动状态。

4.5 广场发现与相机功能

广场页面组合轮播图与双列图片卡片，使用 GridLayoutManager 和不同条目类型承载内容。轮播条目占满一行，其余图片占单列；由于轮播占用第一个位置，图片数据索引需要与适配器位置进行转换。点击图片进入横向 ViewPager2 图片浏览页。该布局属于双列网格，不能仅因视觉效果相似就认定其为等高规则之外的瀑布流。

发现页提供分类、主题播单和话题入口，分类详情通过不同列表页展示热门和最新内容。搜索页把关键词提交至接口并显示视频结果。搜索请求需要考虑空关键词与快速重复提交；话题区域不能假定服务端始终返回固定数量的数据。

CameraX 模块用于头像拍照，通过相机权限检查、预览和拍照获取图片，再交由资料修改流程处理。它不等同于短视频录制与上传模块，报告中的功能范围应与实际页面入口一致。拒绝权限、未选择图片以及上传失败均需要提供可恢复的提示。

4.6 本地浏览记录

Room 实体 UserLookRecord 对应 user_look_record 表。每次访问视频详情后，按用户编号和视频编号查找记录；已存在时更新时间，不存在时插入。读取时按浏览时间倒序排列，支持对选中视频批量删除。数据库访问在工作线程执行，结果通过主线程 Handler 返回界面。

字段	类型	作用
id	int	自增主键
user_id / video_id	String / int	区分用户及视频
cover / title / label	String	封面、标题、标签
duration	String	视频时长展示
view_time	long	浏览时间与倒序排序

五　实验结果与分析

5.1 功能核对与测试设计

从工程结构和代码调用链可以确认，系统已组织首页浏览、独立播放、用户互动及浏览记录等功能路径。功能运行结果应结合设备操作与服务端响应记录，以下用例给出验收方法，实际结果栏在执行后填写，不以静态代码检查代替运行通过。

测试项目	操作与预期结果	实际结果
主界面导航	依次切换四入口，返回首页核对列表位置	待实测
推荐与日报	进入两类列表；刷新替换、加载追加	待实测
游客播放	未登录点击视频，详情与媒体正常加载	待实测
连续切换视频	播放页打开另一个视频，内容与评论同步切换	待实测
播放生命周期	切后台、返回及关闭页面，核对暂停与释放	待实测
验证码登录	使用有效测试账号登录，个人页刷新	待实测

测试项目	操作与预期结果	实际结果
未登录互动	点赞、收藏、评论被提示登录	待实测
已登录互动	执行点赞收藏及取消，与服务端状态一致	待实测
评论提交	输入有效评论，检查新增及重复提交	待实测
浏览记录	重复查看同一视频，时间更新且不重复显示	待实测
记录删除	勾选记录删除，重进页面后仍已移除	待实测
异常与权限	断网、空数据、拒绝相机权限，提示且可恢复	待实测

测试环境记录：设备型号_____；Android 版本_____；应用版本_____；网络条件_____；执行日期_____。应分别使用游客与测试账号操作，截图时隐藏手机号、令牌和其他个人信息。

5.2 界面截图与运行结果记录

界面截图用于证明页面和业务状态。建议使用同一设备、同一尺寸截取，裁除无关桌面内容，保留必要状态栏。每组图片应有图号与说明；若使用前后对照图，需保证账号和视频对象一致。下列位置可直接替换为实际截图。

5.2 界面截图与运行结果记录续

【插图位置】并排插入两张手机截图：首页推荐列表与发现分类页。保留顶部栏目、视频卡片和底部导航，用于说明内容入口及列表布局。

图 5-1　首页与内容发现界面

【插图位置】并排插入同一视频的简介页与评论页截图，包含播放器、标题、互动按钮和评论列表。应能核对两页对应同一视频。

图 5-2　视频播放与评论界面

【插图位置】并排插入验证码登录页与登录后的个人中心截图，隐藏完整手机号及验证码，保留昵称和收藏、浏览记录入口。

图 5-3　用户登录与个人中心

【插图位置】并排插入收藏列表和浏览记录页面；记录删除可用前后对照小图补充，体现数据重新进入页面后的持久化效果。

图 5-4　收藏与浏览记录

【插图位置】插入广场轮播及双列图片界面；如需展示相机模块，另附头像拍照预览图。相机图只展示头像采集功能。

图 5-5　广场与头像拍照

运行结果记录：填写成功完成的流程、出现的异常、复现步骤及对应图号。对于未执行的用例保留待测标记；对于失败项记录具体提示或日志摘要，修复后再追加复测结论。

5.3 问题分析与改进方向

5.3.1 结构设计的特点

组件化将内容浏览、用户与媒体等职责拆分，有利于定位功能边界。公共列表基类复用分页与状态页，减少多个业务列表重复编写相同交互。Room 保存浏览索引，使用户回看历史时无需仅依靠远程历史接口。Java 与 Kotlin 共存也为分阶段迁移提供了条件，但前提是状态源和接口语义保持一致。

5.3.2 当前实现需要关注的问题

第一，Java 与 Kotlin 播放相关实现并存，部分弹窗仍使用旧 ViewModel，容易造成状态不同步。应梳理真实路由入口，统一播放页与子页面共享的状态对象，再逐步清理旧实现。

第二，网络回调需要在访问响应字段前判空，并区分 HTTP 失败、业务失败和空数据。统一请求封装应避免输出包含令牌及用户资料的完整日志，尤其在发布版本中控制日志内容。

第三，分页失败与重复点击需要增加一致性处理。页码只有在请求成功后才能稳定推进；点赞收藏若采用乐观更新，应在失败时恢复状态。单纯修改界面数字并不能证明服务端已保存结果。

第四，播放器返回前台的恢复、页面复用时推荐列表的刷新以及评论列表滚动位置均值得专项验证。释放资源解决的是退出页面后的占用问题，恢复策略则需要结合用户主动暂停状态进行设计。

第五，浏览记录当前通过“先查再写”避免一般情况下的重复记录，但并发写入仍可能产生竞争。后续可为用户与视频组合设置唯一约束，并以事务或数据库冲突策略保证一致性。

5.3.3 后续优化安排

优先补齐网络判空、互动失败恢复和播放生命周期验证，再处理搜索防抖、

收藏分页、话题数量不足时的布局容错。进一步优化可加入自动化接口测试与关键页面回归测试，使用性能分析工具检查长列表图片加载、播放器释放和数据库实例管理。分享和视频发布应作为新增需求单独设计、实现与验收。

六　小结与心得体会

本课程设计围绕 Android 短视频客户端，将内容列表、媒体播放、登录互动和本地记录组织在同一工程中。通过梳理需求与代码，我对智能终端应用的理解从单一页面控件扩展到页面状态、数据访问和资源管理之间的协作。一个页面能够显示数据只是起点，切换页面、账号变化和网络失败之后仍保持一致，才是完整应用需要解决的问题。

在架构方面，组件划分帮助我建立了职责意识。首页负责组织推荐与日报，媒体模块承担播放和互动，公共层提供路由、基类及用户管理。这样的边界使查找功能更清楚，也让我认识到公共封装应服务于真实复用；若把所有业务逻辑放入基类，反而会增加理解和修改成本。

在数据处理方面，网络响应、界面状态与本地存储各有不同的生命周期。LiveData 能通知界面刷新，但不能替代服务端确认；加密偏好存储能保护本地会话资料，但令牌是否有效仍由接口判断；浏览记录能保存回看入口，却不意味着媒体已经离线保存。明确这些区别，有助于避免功能说明与实际能力不一致。

视频播放让我更加关注生命周期。播放器准备、暂停、恢复与释放需要与页面行为配合，singleTask 复用页面时还必须同步更新关联数据。后续我会通过连续切换视频、后台返回和异常媒体地址等场景验证实现，而不只观察一次正常播放。

本次报告整理也让我认识到测试记录的重要性。功能存在于代码中与功能在设备上稳定运行是两个层面，实验结果应由截图、操作步骤和实际现象支撑。今后的学习中，我会先把核心流程拆成可验证的小任务，及时记录问题与修改依据，再通过复测确认改动效果，逐步提高开发与技术表达能力。